HITAISH KUMAR

9861082561 | hitaishkumar7587@gmail.com | linkedin.com/in/davythedev

Skills

Languages: Go, Python, TypeScript, Node.js, SQL
Distributed Systems: Event-Driven Architecture, Message Queues, Concurrent Workers, Async Processing, Rate Limiting, Backpressure, Idempotency, Retry & Failure Handling, DLQ
AWS: ECS, Fargate, EC2 Auto-Scaling Groups, S3, SQS, ALB, CloudWatch, CDK
Backend: Go (Gin/Echo), REST APIs, Background Workers, Durable Job Queues, API Design, OpenAPI
Databases: PostgreSQL, ClickHouse, Redis, MySQL, MongoDB; Schema Design, Indexing, Partitioning, Query Optimization
Messaging & Infra: RabbitMQ, Kafka, Docker, Kubernetes
Reliability & Observability: OpenTelemetry, Prometheus, Grafana, CloudWatch, pprof Profiling, Structured Logging, Load Testing
Security: AES-256-GCM Encryption, Cedar Authorization, Multi-Tenant Isolation
Practices: System Design, Design/Code Reviews, CI/CD, Testing, Operational Excellence, On-Call Production Support

Experience

Senior Software Engineer (R&D) **Aug 2024 – Present**
KFINTECH *Bhubaneswar, Odisha*

- **Multi-Tenant Notification Orchestration Platform** — (**600M+ communications**, **10M+ users**)
  - Owned the end-to-end design and delivery of a **multi-service, event-driven platform** in Go (Publisher API, Distributor, Consumer, Webhook, Campaign Manager, Flusher) that executes omni-channel campaigns (Email/SMS/WhatsApp) with conditional flow routing and failover paths.
  - Architected a **high-throughput consumer engine** using goroutine-per-message concurrency with **semaphore-bounded workers (1000 in-flight)** and QoS prefetch matching, sustaining large-scale message processing without memory exhaustion.
  - Designed an **async distribution pipeline** on RabbitMQ using **per-campaign vhost isolation**, token-bucket rate limiting, and self-terminating **on-demand Fargate tasks**, eliminating noisy-neighbor contention and idle compute cost.
  - Built a **durable, exactly-once campaign job queue** on Postgres (River) that atomically transitions campaign state and enqueues work in a single transaction, with ECS task-slot admission control and automatic snooze/retry under capacity limits.
  - Engineered a **fault-tolerant delivery lifecycle** — publisher confirms, per-campaign dead-letter queues, per-goroutine panic recovery with alerting, auto-reconnect self-healing, and lease-based quota with graceful shutdown on exhaustion.
  - Implemented **AES-256-GCM request/response encryption middleware** and **Cedar-based per-resource authorization** across all APIs, securing multi-tenant customer data.
  - Designed **schema-per-workspace multi-tenancy** on PostgreSQL and **per-workspace ClickHouse** analytics; optimized indexes, ORDER BY keys, and partitioning to cut data-intensive dashboard latency by **40%**.
  - Modeled an append-only **ClickHouse ReplacingMergeTree staging design** (flag-based state, no deletes) enabling high-write-throughput wait-time evaluation and reverse-feed reconciliation from vendor webhooks.
  - Dove deep into ClickHouse merge behavior under benchmark load (**1.09M rows / 628 × 65MB files / 48GB per batch**); root-caused single-thread merge blockage and tuned merge pool, part limits, and memory ratios to keep part counts healthy and prevent OOM on constrained instances.
  - Instrumented all services with **OpenTelemetry tracing, Prometheus metrics, Grafana, CloudWatch, and pprof profiling**, and auto-generated per-service CloudWatch queries via a pre-commit pipeline, supporting **99.9% uptime** for R&D launches.
  - Reduced infrastructure cost through **tiered EC2 Auto-Scaling Groups** by workspace plan and pay-per-use Fargate tasks that self-terminate on idle, aligning compute spend with actual demand.
  - Built React/Next.js operational dashboards (TanStack Query) for campaign monitoring, audit trails, and large-volume delivery reporting.

Software Engineer (Golang) **Jun 2023 – Aug 2024**
KFINTECH *Bhubaneswar, Odisha*

- **Investment Analytics Platform** — (**10M+ investors**)
  - Designed and implemented **high-throughput Go APIs** serving **100K+ concurrent users** on AWS ECS for large-scale financial workloads.
  - Owned SQL data modeling, indexing, and partitioning strategies for large datasets, improving query performance for operational and investor-facing workloads.
  - Designed concurrent **Go worker pools** for background reconciliation and document-processing jobs with robust error handling and retry mechanisms.
  - Deployed and operated scalable backend workloads using AWS ECS, S3, Docker, and CDK.
- **Statement of Account Generation Platform** — (**13M documents/hour**)
  - Re-architected document-generation pipelines with **Go concurrency, worker pools, and S3-backed object storage**, increasing throughput by **10x** to approximately **13M documents/hour**.
  - Isolated document generation from downstream storage/delivery via async processing flows, with retry and failure handling for high-volume resilience.
  - Performed large-scale load testing with JMeter to identify throughput, concurrency, and resource bottlenecks.

Projects

- **Distributed Search & Log Analytics Platform** — Independent Distributed Systems Project
  - Built a distributed search platform on **OpenSearch, Go, Kafka, and S3** with concurrent ingestion workers supporting batching, backpressure, retries, and dead-letter handling.
  - Designed index mappings, shard/replica strategies, and hot/warm lifecycle policies balancing **search latency, indexing throughput, storage, and cost**; documented capacity-planning models for shard and scaling thresholds.
  - Added Prometheus/OpenTelemetry instrumentation and load-tested ingestion/search to evaluate latency, CPU, memory, and storage trade-offs; designed recovery for worker crashes, duplicate events, and downstream unavailability.
- **KiteRunner CLI** — Open-source Algo Trading Terminal (Go)
  - Built a concurrent Go CLI for real-time market-data retrieval using parallel fetchers, applying low-latency techniques to reduce blocking across independent data sources.
- **AI-Powered API Scaffolder** — Side Project
  - Built a developer tool generating type-safe Node.js and Go API boilerplate via LLM APIs, focusing on structured code generation, consistency, and secure validation of generated code.

Education

Veer Surendra Sai University of Technology **Odisha**
Electrical Engineering, 8.2 GPA Sep 2019 – Apr 2022